



Building an Authentication App with React

A beginner-friendly guide covering everything from project setup to a fully styled multi-page app with unit tests and GitHub deployment.

React 18

TypeScript

Vite

Vitest

React Router v7

CSS Modules

Written by Claude · June 2026

Table of Contents

1. What Are We Building?
2. Step 1 — Project Setup with Vite
3. Step 2 — Installing Dependencies
4. Step 3 — Configuring Vitest for Testing
5. Step 4 — The Mock API
6. Step 5 — The Login Page Component
7. Step 6 — Writing Unit Tests for the Login Page
8. Step 7 — Styling the Login Page
9. Step 8 — Adding Navigation Links
10. Step 9 — Setting Up Routing with React Router
11. Step 10 — The Sign Up Page
12. Step 11 — The Home Page (After Login)
13. Step 12 — The Forgot Password Page
14. Step 13 — Setting Up Git and Pushing to GitHub
15. Final Project Structure
16. Key Concepts Summary

 [How to read this guide](#)

Each step builds on the previous one. Code examples show exactly what was written, and explanations focus on the *why* — not just the *what*. By the end you'll have a working app with 58 passing unit tests.

What Are We Building?

Before writing a single line of code, it's important to understand the end goal. We're building a small but complete authentication app — the kind of thing that sits at the front of almost every real-world web application.

Pages

- **Login Page** — email, password, and "Remember Me" checkbox. Validates input and navigates to the Home page on success.
- **Sign Up Page** — full name, email, password, and confirm password. Shows success or error feedback.
- **Home Page** — shown after a successful login. Has a nav bar with a Log Out button.

Features

- Client-side form validation with helpful error messages
- Loading state on the submit button while the "API" is working
- Navigation between pages using React Router
- Styling with CSS Modules (each component has its own stylesheet)
- 58 unit tests covering rendering, validation, interactions, API calls, and navigation
- All code committed and pushed to a public GitHub repository

💡 What is a "mock API"?

A real app would talk to a server to check your password. Since we're building a frontend-only project, we simulate that server call with a JavaScript function that waits 500 milliseconds and then returns a fake success or failure. This keeps the focus on learning React without needing a backend.

Every React project needs a build tool — software that bundles your code and runs a local development server. We use **Vite** (pronounced "veet"), which is fast, modern, and extremely popular in the React community.

Why Vite instead of Create React App?

Create React App (CRA) was the go-to tool for years, but it's slow and no longer actively maintained. Vite starts in milliseconds and rebuilds instantly when you save a file, which makes development much more enjoyable.

The project was created manually

Because the project directory already existed, we created the files by hand instead of running the Vite scaffolding command. Here's what each key file does:

```
claude_test/ |— index.html ← The single HTML file. Vite injects your JS here.
              |— package.json ← Lists all dependencies and run scripts. |— vite.config.ts ←
              Tells Vite how to build and test the project. |— tsconfig.json ← TypeScript
              configuration. |— src/ |— main.tsx ← Entry point — mounts the React app into
              the HTML. |— App.tsx ← Defines the routes (which page to show at which URL).
```

The package.json scripts

These are the commands you'll use every day as a developer:

```
"scripts": {
  "dev": "vite", // Start the local dev server at http://localhost:5173
  "build": "tsc && vite build", // Compile TypeScript and bundle for production
  "test": "vitest", // Run tests in watch mode (re-runs when files change)
  "test:run": "vitest run" // Run tests once and exit (used in CI pipelines)
}
```

TypeScript vs JavaScript

We chose TypeScript (TS) over plain JavaScript (JS). TypeScript adds type annotations that catch mistakes while you're writing code — before you even run it. For example, if a function expects an email string and you accidentally pass a number, TypeScript shows an error immediately. Files

use `.ts` and `.tsx` extensions. The `x` in `.tsx` means the file contains JSX (HTML-like syntax inside JS).

3 Step 2 — Installing Dependencies

In the JavaScript world, "dependencies" are packages of code written by others that you include in your project. They're managed by **npm** (Node Package Manager) and listed in `package.json`.

Production dependencies

Code that ships to the user's browser:

Package	What it does
<code>react</code>	The core React library — the engine that builds UIs from components.
<code>react-dom</code>	Connects React to the browser's DOM (the real web page).
<code>react-router-dom</code>	Handles navigation between pages without a full page reload.

Development dependencies

Code only used during development and testing — never shipped to users:

Package	What it does
<code>vitest</code>	The test runner. Finds and executes all <code>*.test.tsx</code> files.
<code>jsdom</code>	Simulates a browser environment so tests can render React components.
<code>@testing-library/react</code>	Provides <code>render()</code> , <code>screen</code> , and <code>waitFor()</code> for testing components.
<code>@testing-library/user-event</code>	Simulates real user actions like typing and clicking.
<code>@testing-library/jest-dom</code>	Adds matchers like <code>toBeInTheDocument()</code> and <code>toBeDisabled()</code> .
<code>typescript</code>	The TypeScript compiler.
<code>@vitejs/plugin-react</code>	Allows Vite to process JSX syntax.

```
npm install react react-dom react-router-dom
```

```
npm install -D vitest jsdom @testing-library/react @testing-library/user-event @testing-library/dom
```

4 Step 3 — Configuring Vitest for Testing

Out of the box, Vitest doesn't know that our tests run inside a browser-like environment. We need to tell it two things: use `jsdom` to simulate the browser, and load the `jest-dom` matchers before every test file.

`vite.config.ts`

```
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true, // Makes describe(), it(), expect() available without importing
    environment: 'jsdom', // Simulates a browser (window, document, etc.)
    setupFiles: ['./src/setupTests.ts'], // Runs this file before every test
  },
})
```

`src/setupTests.ts`

```
import '@testing-library/jest-dom'
// This line adds all the custom matchers (toBeInTheDocument, toBeChecked, etc.)
// to Vitest's expect() function.
```

💡 What is a "matcher"?

In a test you write `expect(something).toBeInTheDocument()`. The `toBeInTheDocument` part is a matcher — it checks a specific condition and throws an error if it fails. React Testing Library adds matchers that are specifically designed for checking HTML elements in the DOM.

5 Step 4 — The Mock API

Before building the UI, we created the "API layer" — the code responsible for talking to a server. Since we don't have a real server, both functions simulate a network call using `setTimeout`.

`src/api/auth.ts`

```
export async function loginUser(email: string, password: string) {
  // Wait 500ms to simulate a real network request
  await new Promise((resolve) => setTimeout(resolve, 500))

  // Hardcoded valid credentials for demo purposes
  if (email === 'admin@example.com' && password === 'password123') {
    return { token: 'mock-token' }
  }

  // If credentials don't match, throw an error just like a real API would
  throw new Error('Invalid credentials')
}

export async function signUpUser(name: string, email: string, password: string) {
  await new Promise((resolve) => setTimeout(resolve, 500))

  // Simulate an "email already taken" scenario
  if (email === 'existing@example.com') {
    throw new Error('An account with this email already exists')
  }

  return { token: 'mock-token' }
}
```

Why is this in a separate file?

Separating API calls into their own module (`src/api/auth.ts`) is a best practice called **separation of concerns**. The component shouldn't need to know *how* the login works — it just calls a function and gets a result back. This also makes testing much easier: in tests, we can replace the real function with a fake one (called a "mock") without changing any component code.

 **What is async/await?**

Network requests take time. `async` marks a function that does something time-consuming, and `await` pauses execution until that thing finishes — without freezing the entire browser. Without `async/await`, you'd need to use callbacks or `.then()` chains, which are much harder to read.

A React component is a JavaScript function that returns JSX — HTML-like syntax that describes what should appear on screen. The Login page component manages its own state (the values the user has typed) and handles form submission.

State — what the component "remembers"

```
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
const [rememberMe, setRememberMe] = useState(false)
const [errors, setErrors] = useState({}) // Validation error messages
const [status, setStatus] = useState('idle') // 'idle' | 'loading' | 'error'
const [apiError, setApiError] = useState('') // Error message from the API
```

Validation — checking input before calling the API

```
function validate(email, password) {
  const errors = {}

  if (!email.trim())
    errors.email = 'Email is required'
  else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.trim()))
    errors.email = 'Invalid email address'

  if (!password)
    errors.password = 'Password is required'
  else if (password.length < 6)
    errors.password = 'Password must be at least 6 characters'

  return errors
}
```

Submission flow

1. User clicks "Log In"
2. `validate()` runs — if there are errors, show them and stop
3. Set `status = 'loading'` — the button shows "Logging in..." and is disabled
4. Call `loginUser(email, password)` and wait for the result
5. **Success** → navigate to `/home`

6. **Failure** → set `status = 'error'` and show the error message

7

Step 6 — Writing Unit Tests for the Login Page

Unit tests verify that a single, isolated piece of your app works correctly. They're like a checklist you run automatically every time you make a change — if something breaks, the test tells you immediately.

How a test is structured

```
it('shows "Email is required" when submitting with empty email', async () => {  
  // 1. ARRANGE — Set up the user and render the component  
  const user = userEvent.setup()  
  render(<LoginPage />)  
  
  // 2. ACT — Simulate the user doing something  
  await user.click(screen.getByRole('button', { name: /log in/i })))  
  
  // 3. ASSERT — Check that the expected result happened  
  expect(screen.getByText('Email is required')).toBeInTheDocument()  
})
```

Every test follows the **Arrange** → **Act** → **Assert** pattern.

Mocking the API in tests

We don't want tests to wait 500ms for a real (or fake) API call. Instead, we replace `loginUser` with a controllable fake using `vi.mock()`:

```
vi.mock('../api/auth') // Replace the entire module with auto-generated mocks  
  
const mockLoginUser = vi.mocked(authModule.loginUser)  
  
// In a test, control what the mock returns:  
mockLoginUser.mockResolvedValueOnce({ token: 'mock-token' }) // Success  
mockLoginUser.mockRejectedValueOnce(new Error('Invalid credentials')) // Failure
```

Test groups for the Login page

Group	Tests	What's verified
A — Rendering	4	All form elements appear when the page loads

B — Validation	6	Error messages for invalid or missing input
C — Interactions	3	Typing updates fields, checkbox toggles
D — Submission	6	API calls, loading state, navigation, error display
E — Edge Cases	2	Email whitespace trimming, error clearing on retry
F — Links	5	"Forgot password?" and "Sign up" links present

⚠️ A mistake we caught and fixed

When we added the 🚩 emoji to error messages (e.g. "🚩 Invalid credentials"), existing tests that used `getByText('Invalid credentials')` started failing — because the full text was now "🚩 Invalid credentials". The fix was to switch to a regex: `getByText(/Invalid credentials/)`, which matches any text containing that phrase. Bugs like this are exactly what tests are designed to catch.

CSS Modules are a way to write CSS where every class name is automatically made unique to its component. This prevents one component's styles from accidentally affecting another — a very common problem in large projects.

How CSS Modules work

```
// LoginPage.module.css → .button { ... }  
// In LoginPage.tsx:  
import styles from './LoginPage.module.css'  
// Usage:  
<button className={styles.button}>Log In</button>  
// In the browser, the class becomes something like: _button_8e3085  
// This ensures no two components can ever have the same CSS class name.
```

Design decisions

- **Purple gradient background** — creates visual depth and makes the white card stand out.
- **White card with shadow** — "lifts" the form off the background using `box-shadow`.
- **Focus ring on inputs** — a subtle glow when an input is selected (accessibility + polish).
- **Red border on error** — invalid inputs get a red `border-color` so users know exactly what to fix.
- **Gradient button** — consistent with the background; the button visually belongs to the page.

Two links were added to the Login page to help users navigate:

- **"Forgot password?"** — placed right-aligned on the same row as the Password label using flexbox (`justify-content: space-between`).
- **"Don't have an account? Sign up"** — sits below the Log In button, centered, in subtle gray with a purple bold link.

The "Forgot password?" link uses a plain `<a href="#">` tag for now (a placeholder for a future page). The "Sign up" link was later upgraded to a React Router `<Link>` component so it navigates without reloading the page.

💡 `<a>` vs `<Link>`

A regular `<a href="/signup">` tag causes a full page reload — the browser fetches a new HTML document from the server. React Router's `<Link to="/signup">` intercepts the click and updates the URL and page content entirely in JavaScript, with no server request. This is what makes React apps feel instant.

React Router lets us build a **Single Page Application (SPA)** — one HTML file that shows different "pages" by swapping components in and out based on the URL, without ever reloading.

Three files were updated or created

```
// src/main.tsx — wrap everything in BrowserRouter
import { BrowserRouter } from 'react-router-dom'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
)
```

```
// src/App.tsx — define which component to show at each URL
import { Routes, Route } from 'react-router-dom'
export default function App() {
  return (
    <Routes>
      <Route path="/" element={<LoginPage />} />
      <Route path="/signup" element={<SignUpPage />} />
      <Route path="/home" element={<HomePage />} />
    </Routes>
  )
}
```

Testing with MemoryRouter

When testing components that use `<Link>` or `useNavigate`, they need to be wrapped in a Router — otherwise React throws an error. In tests we use `<MemoryRouter>` instead of `<BrowserRouter>` because it works without a real browser URL bar:

```
function renderPage() {
  return render(
    <MemoryRouter initialEntries={['/']}>
      <Routes>
        <Route path="/" element={<LoginPage />} />
        <Route path="/home" element={<div>home-sentinel</div>} />
      </Routes>
    </MemoryRouter>
  )
}
```

```
    </Routes>
  </MemoryRouter>
)
}
```

The Sign Up page follows exactly the same pattern as the Login page — it's a great example of how React components are **reusable patterns**, not one-off solutions.

Additional fields and validation rules

Field	Validation	Error message
Full Name	Required	"Full name is required"
Email	Required + valid format	"Email is required" / "Invalid email address"
Password	Required + min 6 chars	"Password is required" / "Password must be at least 6 characters"
Confirm Password	Required + must match password	"Please confirm your password" / "Passwords do not match"

The "passwords must match" check

```
if (!confirmPassword) {  
  errors.confirmPassword = 'Please confirm your password'  
} else if (confirmPassword !== password) {  
  errors.confirmPassword = 'Passwords do not match'  
}
```

The Sign Up page has 25 unit tests across 5 groups (rendering, validation, interactions, submission, navigation).

After a successful login, the user is automatically redirected to `/home` using React Router's `useNavigate` hook. The Home page acts as a simple dashboard.

How navigation works on success

```
// Inside LoginPage.tsx:
const navigate = useNavigate()

// After API success, instead of showing a message, navigate away:
try {
  await loginUser(email.trim(), password)
  navigate('/home') // ← This replaces the current page with the Home page
} catch (err) {
  setStatus('error')
  setApiError(err.message)
}
```

The Log Out button

```
// Inside HomePage.tsx:
const navigate = useNavigate()

<button onClick={() => navigate('/')}>Log Out</button>
// Clicking this navigates back to the login page
```

Home page contents

- Navigation bar with the app brand and a Log Out button
- "Welcome back! 🙌" heading with a subheading
- Three dashboard cards: Analytics, Settings, and Notifications

The Home page has 7 unit tests covering rendering (A1–A6) and logout navigation (B1).

The "Forgot password?" link on the Login page was originally a dead placeholder (`<a href="#">`). We upgraded it into a full working page at `/forgot-password` with its own form, API call, and success screen.

What was changed and added

- **API** — added `forgotPassword(email)` to `src/api/auth.ts`
- **New component** — `ForgotPasswordPage.tsx` + `ForgotPasswordPage.module.css`
- **New route** — `/forgot-password` registered in `App.tsx`
- **LoginPage link upgraded** — `<a href="#">` replaced with `<Link to="/forgot-password">`
- **15 new unit tests** in `ForgotPasswordPage.test.tsx`
- **LoginPage test F2 updated** — now asserts `href="/forgot-password"`

The new API function

```
export async function forgotPassword(email: string) {
  await new Promise((resolve) => setTimeout(resolve, 500))
  // Any valid email format is accepted (mock - no real DB lookup)
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.trim())) {
    throw new Error('No account found with that email address')
  }
  return { message: 'Password reset link sent' }
}
```

Component behaviour

1. User enters their email and clicks "Send reset link"
2. Client-side validation runs first (required + email format)
3. Button changes to "Sending..." and is disabled
4. **Success** → the form is replaced by a success screen showing "Check your inbox" and the submitted email address, plus a "Back to Log In" button
5. **Failure** → an error alert appears and the button is re-enabled

Success screen (replaces the form entirely)

```

{status === 'success' ? (
  <div role="status">
    <h2>Check your inbox</h2>
    <p>We sent a reset link to <strong>{email.trim()}</strong></p>
    <Link to="/">Back to Log In</Link>
  </div>
) : (
  <form>...</form>
)}

```

Unit tests — 15 tests across 4 groups

Group	Tests	What's verified
A — Rendering	4	Email input, button, heading, "Log in" link all present
B — Validation	3	Required email, invalid format, API not called on failure
C — Submission	6	API called with trimmed email, loading state, success screen, error display, button re-enabled
D — Navigation	2	"Log in" link and "Back to Log In" button both navigate to /

💡 Why replace the form with a success screen?

Once the reset email is sent, there's nothing more for the user to do on this page. Replacing the form prevents accidental re-submission and gives a clear visual signal that the action completed. The user's only next step is checking their email — so we surface that clearly and offer a direct route back to login.

13

Step 13 — Setting Up Git and Pushing to GitHub

Git is a version control system — it tracks every change you make to your code over time. **GitHub** is a website that hosts your Git repository online so it can be shared and backed up.

The basic Git workflow

```
git init                # Create a new repository in the current folder
git add .               # Stage all changed files for committing
git commit -m "Describe the change" # Save a snapshot with a message
git remote add origin <URL> # Link to a GitHub repository
git push -u origin master # Upload commits to GitHub
```

The .gitignore file

Not everything should be committed to GitHub. The `.gitignore` file tells Git to skip certain folders and files:

```
node_modules/ # Hundreds of MB of packages — GitHub users run npm install themselves
dist/         # The built output — always regenerated from source
coverage/     # Test coverage reports
```

Commit history

Commit	What was added
<code>e106b8d</code>	Initial project — login page, tests, CSS styling
<code>205e377</code>	Forgot password link (placeholder)
<code>5aaa1d3</code>	Sign up link
<code>b085cf5</code>	Unit tests for navigation links (F1–F5)
<code>c04d62b</code>	Sign Up page with routing, mock API, and 25 tests
<code>990d64d</code>	Home page with logout navigation and 7 tests

02dfdf3

CLAUDE.md project documentation

b274b93

Beginner guide PDF

121617e

Update credentials to benchmekh@gmail.com in mock API and tests

abfdacc

Forgot Password page — full form, success screen, 15 unit tests



Good commit messages

Each commit message describes *what* changed and *why* it matters, not *how* it was implemented. "Add Sign Up page with routing, mock API, and unit tests" tells you everything you need to know to understand the change without reading the code. This becomes invaluable when debugging something that broke weeks ago.

14

Final Project Structure

```

claude_test/ ├── .gitignore ├── index.html ├── package.json ├──
vite.config.ts ├── tsconfig.json └─ src/ ├── main.tsx ← Entry point, wraps app
in BrowserRouter ├── App.tsx ← Route definitions (/, /signup, /home, /forgot-
password) ├── setupTests.ts ← Loads jest-dom matchers for all tests ├── vite-
env.d.ts ← TypeScript types for Vite ├── api/ | └─ auth.ts ← loginUser(),
signupUser(), forgotPassword() mock functions └─ components/ ├── LoginPage.tsx
← Login form with validation and navigation ├── LoginPage.module.css ├──
LoginPage.test.tsx ← 26 tests (A-F) ├── SignUpPage.tsx ← Sign-up form with 4
fields ├── SignUpPage.module.css ├── SignUpPage.test.tsx ← 25 tests (A-E) ├──
HomePage.tsx ← Dashboard shown after login ├── HomePage.module.css ├──
HomePage.test.tsx ← 7 tests (A-B) ├── ForgotPasswordPage.tsx ← Forgot password
form + success screen ├── ForgotPasswordPage.module.css └─
ForgotPasswordPage.test.tsx ← 15 tests (A-D)

```


React

A JavaScript library for building UIs out of components — functions that return JSX. Each component manages its own state with `useState` and re-renders automatically when state changes.

TypeScript

A superset of JavaScript that adds static type checking. Files use `.ts` / `.tsx` extensions. Types catch bugs at write-time rather than run-time.

Vite

A modern build tool that runs a lightning-fast development server and bundles your code for production. Replaces the older Create React App.

CSS Modules

A system where CSS class names are scoped to the component that imports them, preventing styling conflicts across the app.

React Router

A library that maps URLs to components, enabling multi-page navigation without full page reloads. Key tools: `<BrowserRouter>` , `<Routes>` , `<Route>` , `<Link>` , `useNavigate` .

Vitest + React Testing Library

The testing stack. Vitest runs the tests; RTL renders components and provides queries like `screen.getByRole()`; `userEvent` simulates real user interactions; jest-dom adds DOM-specific assertions.

Git + GitHub

Git is version control (local history of every change). GitHub is the remote host. Together they give you a complete backup, a collaboration platform, and a record of every decision.

Total test count: 73 passing tests

File	Tests	Coverage area
LoginPage.test.tsx	26	Rendering, validation, interactions, API, navigation, links
SignUpPage.test.tsx	25	Rendering, validation, interactions, API, navigation
HomePage.test.tsx	7	Rendering, logout navigation
ForgotPasswordPage.test.tsx	15	Rendering, validation, API integration, navigation



You built a real authentication flow!

A login page, a sign-up page, a forgot password page, a home dashboard, client-side routing, CSS-styled components, a mock API, and 73 unit tests — all committed to GitHub. That's a solid foundation any real project would be proud of.

React

TypeScript

Vite

Vitest

React Router

CSS Modules

Git

GitHub